

Gate memo — what my experiments say about "≥90% at n=50"

Yogya Mehrotra · Relling take-home · every number here can be regenerated from the README

What I built to answer this

A seeded, exactly-replayable simulation of the stacking cell (UR5e + 2F-85 in MuJoCo): a success judge tested against ten hand-built scenes, a harness that records every episode and produces a one-command report, and four policies of different quality. One extra tool: a "reference policy" with a dialed success rate. My real policies all score under 10%, but the gate's mistakes happen near 90% — the reference policy is how I test that region. It is an instrument, not a robot, and is labeled as such everywhere it appears.

Findings (each from an experiment I ran, not a formula I copied)

- 1. Fifty episodes tells you less than it feels like.** My baseline scored 10% at n=50. At n=1000, its real rate is 5.3% — the 50-episode number was double the truth, purely from which episodes came up. The give-or-take range I report with every rate (the 95% confidence interval, here 4–21%) contained the truth the whole time — which is why I never report a rate without one.
- 2. For any policy near 90%, the gate is close to a coin flip.** I ran the full gate 200 times at each quality level using the reference policies, with the textbook predictions written into my journal before looking at results. The experiment, a 20,000-run coin-flip simulation, and the exact math agree:

true rate	gate passes (measured)	gate passes (exact theory)
80%	6.0%	4.8%
85%	25.0%	21.9%
88%	41.0%	43.5%
90%	64.5%	61.6%
92%	79.5%	79.2%
95%	96.0%	96.2%

A truly-88% policy passes about 4 times in 10. A policy at exactly 90% fails about 4 times in 10. The cause is structural: the passing line ($45/50 = 90\%$) sits exactly on the target, with no margin in either direction — so for any policy near the target, the verdict is decided by which 50 episodes came up, not by the policy.

What this sweep proves, stated plainly: the reference policy draws each outcome from a seeded rate and arranges the scene for the real judge to rule, so it checks the measurement chain (judge, gate arithmetic, seeding) against a known truth — it is not an independent physics experiment, and it can only disagree with the coin-flip math if the judge misrules an arranged scene. The error rates stand on the coin-flip model; its one load-bearing assumption, independence, is tested in Finding 5.

- 3. The gate can't see why a policy fails — a failure taxonomy can.** Two of my real policies (v1 skips tipped-over parts; v2 stands them up) both measured 10% at n=50. Their true rates trend apart (5.3% vs 6.6%) and they fail in completely different ways, but even at n=1000 their confidence intervals overlap — the success count never resolves which one is better. The failure-category counts in my per-episode records separate them instantly. A success count can't tell you why a policy fails or whether a change helped.
- 4. The gate certifies a policy under specific test conditions.** The same v1 scores 10% when parts can spawn tipped (20% chance each) and 22% when parts always start upright. Neither number is "the" rate. If the gate's starting conditions don't match the floor's, the certificate quietly changes meaning. (I asked how the real gate sets its starting conditions.)
- 5. The coin-flip math holds on my cell — checked, not assumed.** All the statistics above assume each episode is an independent coin flip: no streaks, no episode influencing the next. I verified that two ways. The scatter of 50-

episode block scores matches what independent flips produce (ratios of observed-to-expected scatter: 0.93–1.17, where 1.0 is a perfect match), and a reference policy dialed to succeed 88.0% of the time measured 88.4% over 3,000 judged episodes, so the pipeline doesn't distort a known rate. On a real cell this needs re-checking — 50 back-to-back runs with one operator could make episodes influence each other, which would quietly break this math (I asked how gate runs are scheduled).

Recommendation

(a) Stop the gate early once the verdict is decided — do this regardless. After 6 failures a policy cannot reach 45/50. Verdicts are identical; you just get them sooner. A clearly bad policy stops after ~6 episodes instead of 50; near-spec policies average 42–47. Bad policies are where reset labor costs most, so this is savings with no statistical downside.

(b) Move the passing line off the target: require $\geq 47/50$. Same episode count; a truly-88% policy now sneaks through 13.5% of the time instead of 43.5%. The cost: a true-92% policy now fails 57% of the time, so pair this with a cheap retest path. On an aerospace floor I take that trade — shipping an under-spec policy costs more than a retest. If the episode budget can double for borderline candidates, $n=100$ with ≥ 93 beats it on both error rates.

Caveat: the best margin depends on what a false pass costs Relling versus a false rejection, and on what happens after a rejection (retesting until pass raises the false-pass rate). I've asked for both; the recommendation is written to be re-tuned when they arrive, not redone.

Where these results stop

- **Grasping is a software attach** (a part within 20 mm of the closing gripper is held perfectly — the packet's sanctioned fallback, taken after a timeboxed attempt at real contact physics). My evaluation cannot see parts slipping in the grip or dropped mid-carry. If a big share of real failures are grasp-related, my failure counts understate reality (I asked what that share is).
- **My judge's tolerances are my own choices** (base within 25 mm of target, under 10° tilt, 2-second settle), calibrated against the simulator's behavior, not a customer's acceptance spec.
- **The 20% tipped-spawn chance is my reading** of "randomized orientations," and it dominates my failure counts (28 of 45 at $n=50$).
- **True rates are still estimates** ($n=1000$, CIs ± 1 –2 points), and everything inherits the usual sim-to-real gaps: no camera noise, no wear, no lighting, one part shape.
- **Physical parameters the packet left open, caught in a final audit:** cylinder mass 42 g (simulator default density, not chosen), slide friction 1.0 (default), rolling friction 0.001 (picked to stop endless rolling, not measured). The work surface is an infinite plane — a part can never be lost off a table edge. Spawn region x 0.35–0.60 m, y ± 0.25 m, parts ≥ 8 cm apart (spacing my gripper needs; a real cell may not grant it). Episode time limits are mine (60 s v1, 120 s v2) and define the "timeout" category. The judge treats an upside-down cylinder as upright — harmless only because the part is symmetric.

(The stretch experiment — predict-then-test with 10% larger parts — is written up in the README's Stretch section, next to its regeneration command.)

With another week

In priority order: (1) slow the arm's final approach and pause after release — collapse counts say placement is the most valuable fix; (2) re-run the gate audit with Relling's real cost ratio and retest protocol, so the margin is tuned rather than chosen; (3) make v2's tipped-part flip reliable; (4) the chunked-action stretch goal, skipped this round to protect the evaluation work.